

4강. 도커와 컨테이너



한국공학대학교
TECH UNIVERSITY OF KOREA

이영곤



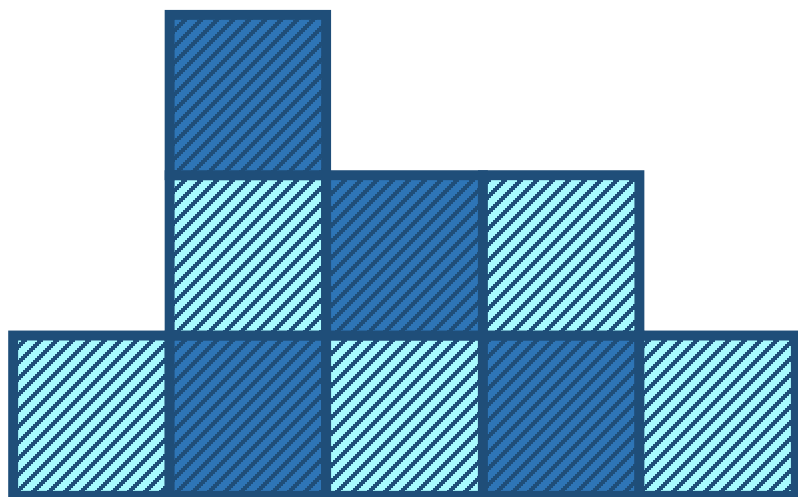
01. Container 소개



» Container



컨테이너란



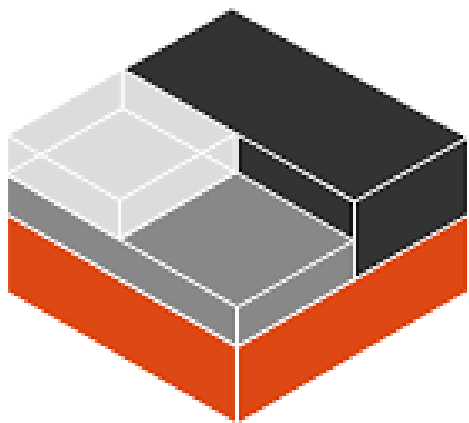
컨테이너

프로그램이 작동하기 위한 최소한의
요소들을 묶어 패키징 한 OS이며
독립적인 배포와 실행을 가능하게 하는
일종의 가상머신

» Linux Container



리눅스 컨테이너란



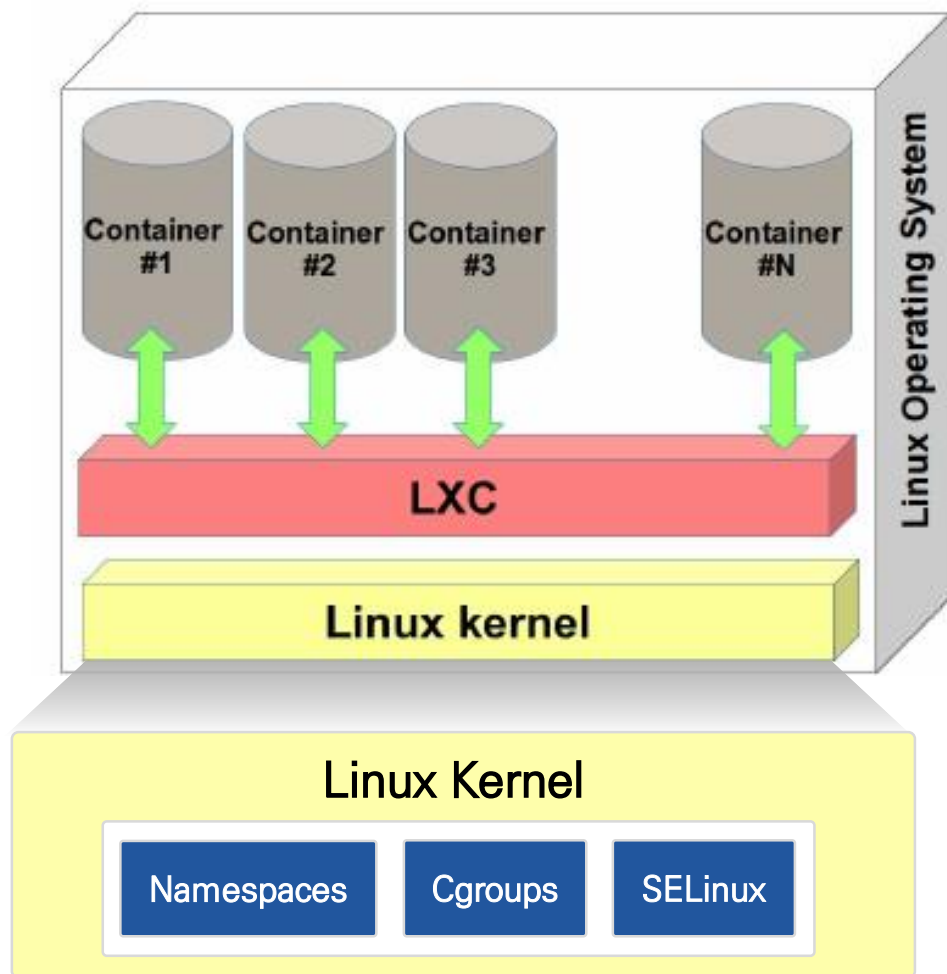
LXC

LXC

LXC(Linux Containers)는 단일 컨트롤
호스트 상에서 여러 개의 고립된 리눅스
시스템(컨테이너)들을 실행하기 위한 운영
시스템 레벨 가상화 방법

» LXC 기술

출처: Grep Haxs



정의

- ✓ 컨테이너를 관리하는 도구들의 모음
- ✓ 리눅스 커널에서 컨테이너에 관한 기능들을 접근하기 위한 사용자 인터페이스

특징

- ✓ 운영체제 수준의 가상화
- ✓ 빠른 실행 속도와 효율성
- ✓ 높은 이식성

» LXC 기술

주 사용 영역

- ✓ 경량 VM 대체 (system container)
 - Docker처럼 프로세스 단위가 아니라 OS 단위 격리
- ✓ 개발/테스트 환경
 - 빠르게 리눅스 환경 여러 개 띄울 때
- ✓ 온프레미스 서버 관리
 - VM보다 가볍고 빠른 환경 필요할 때

역할

- ✓ Docker 이후 변화
 - 애플리케이션 컨테이너
→ Docker가 장악
 - 오케스트레이션 → Kubernetes
 - LXC = OS 컨테이너
 - Docker = 앱 컨테이너

장점

- ✓ VM보다 가벼움
- ✓ Docker보다 완전한 OS 환경
- ✓ systemd, SSH 등 그대로 사용 가능
- ✓ 멀티 테넌시 환경에 적합

» LXC 기술

도커와 차이점

구분	LXC	Docker
목적	OS 격리	앱 실행
실행 단위	리눅스 시스템	단일 프로세스
init(systemd)	있음	없음
SSH 접속	가능	일반적으로 없음
컨테이너 수명	장기 실행	짧고 재생성
이미지 개념	약함	강력 (Dockerfile)
DevOps 친화성	낮음	매우 높음
Kubernetes	거의 안씀	표준
개념	컨테이너로 만든 VM	컨테이너로 실행하는 앱

» Linux Container

Kernel

- ✓ 리눅스 커널에서 컨테이너를 실행 시키는데 중요한 기능을 하는 컴포넌트

Cgroups

컨테이너에 필요한 자원(CPU, 메모리, 블록I/O, 네트워크 등)을 할당

Namespaces

생성된 컨테이너를 다른 컨테이너와 격리 시키는 역할

SELinux

커널 수준의 보안을 제공하여 커널을 보호하는 역할

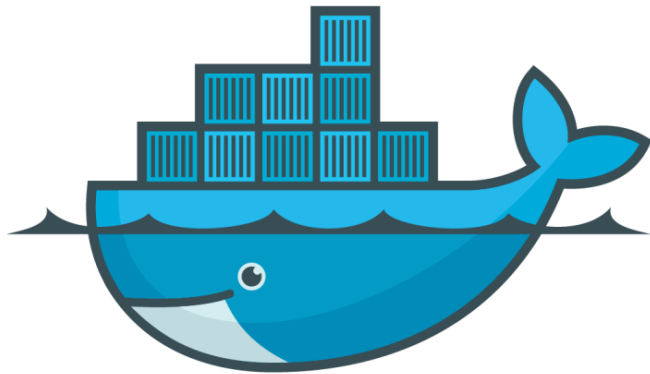
02. Docker 소개



» Docker



도커란



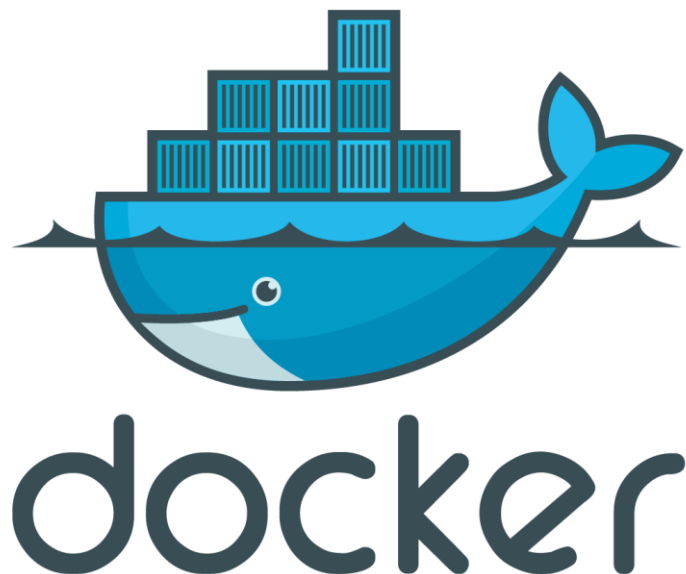
docker

도커

애플리케이션과 실행환경을 포함한
컨테이너를 관리 및 운영하는
오픈 소스 가상화 플랫폼

» Docker

도커 특징

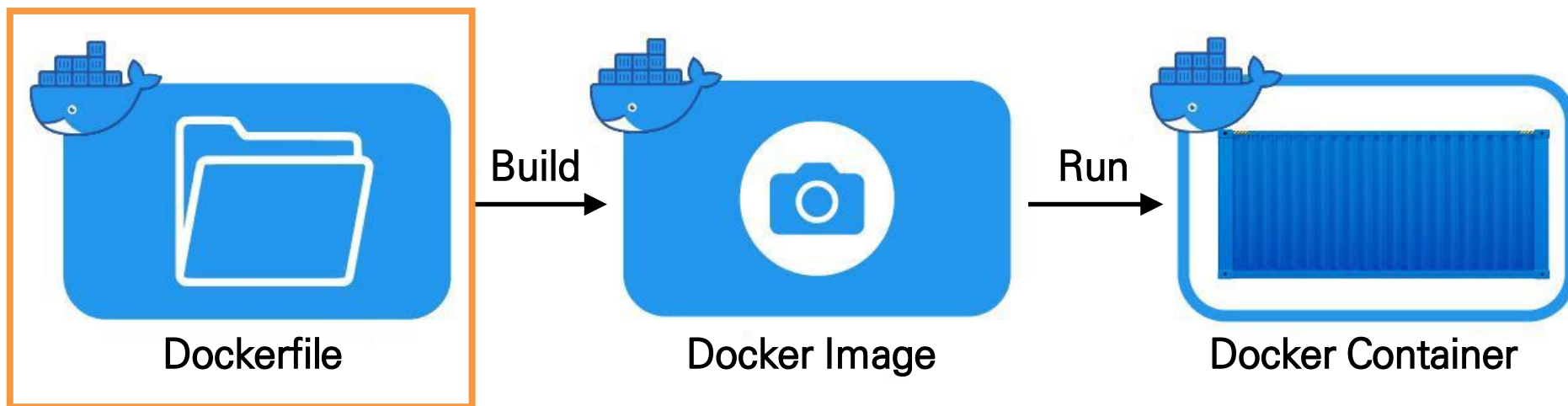


- ✓ 빠른 실행 속도
- ✓ 손쉬운 배포
- ✓ 가벼운 용량
- ✓ 애플리케이션의 독립성
- ✓ 높은 보안성 보장
- ✓ 낮은 오버헤드
- ✓ 이미지 생성 및 공유

» Dockerfile

출처: JFrog

도커 파일



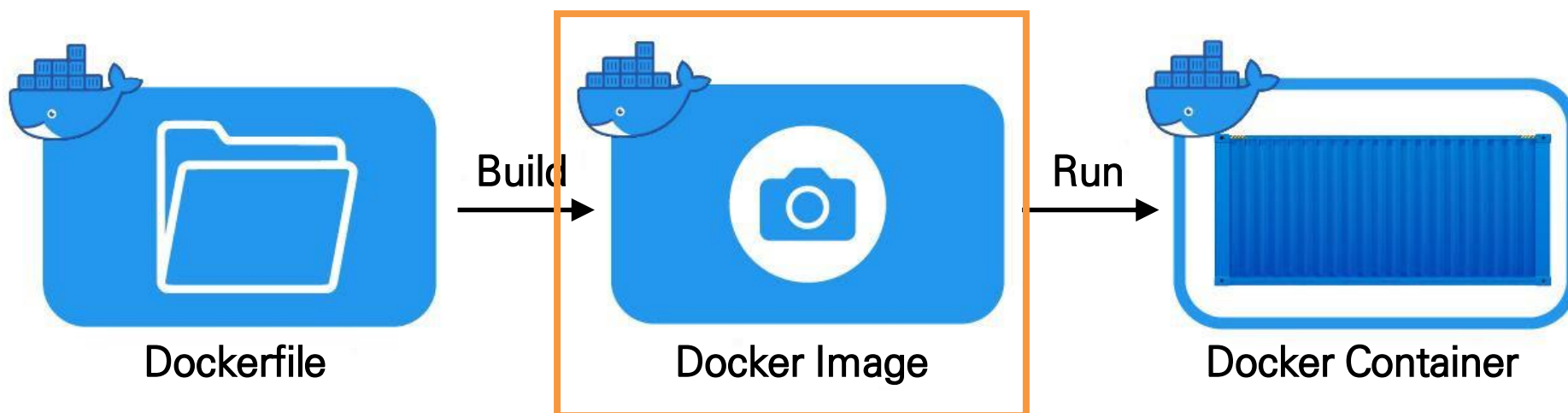
Dockerfile

도커 파일은 도커 이미지를 생성하기 위한 설정 파일이며 다양한 도커 파일 명령어를 사용하여 작성

» Docker Image

출처: JFrog

도커 이미지



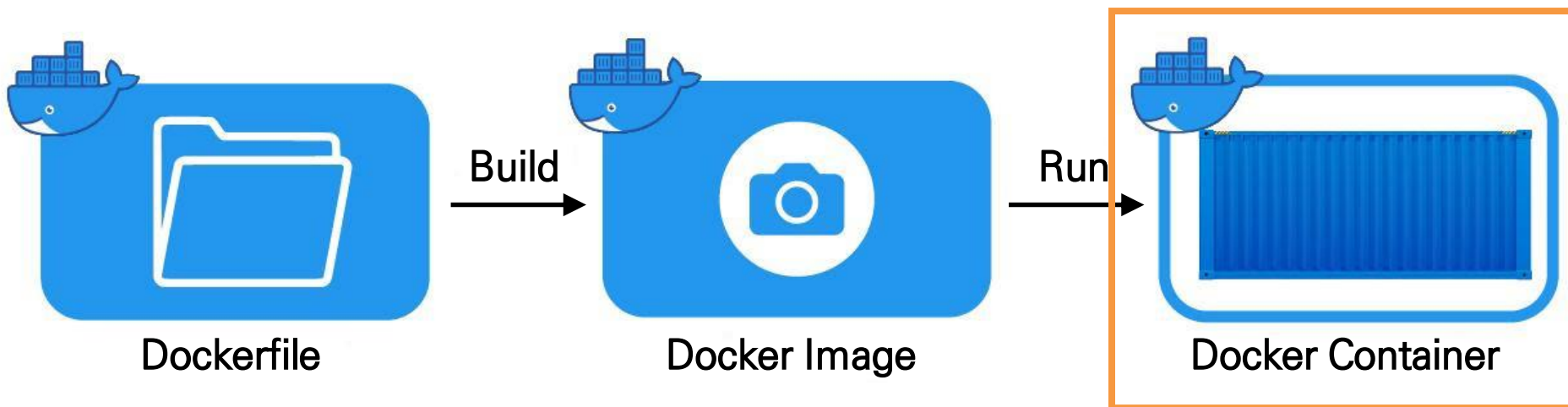
✓ Docker Image

도커 이미지는 컨테이너 실행에 필요한 파일과 설정 값 등을 포함하고 있으며 도커 파일을 빌드하여 생성

» Docker Container

출처: JFrog

도커 컨테이너



✓ Docker Container

도커 컨테이너는 격리된 공간에서 애플리케이션을 동작시킨 상태이며 도커 이미지를 Run하여 도커 컨테이너를 실행

» Dockerfile 작성

도커 파일 예시 (1/3)

```
FROM tomcat:9-jre8-alpine
WORKDIR /usr/local/tomcat
COPY server.xml ./conf
RUN rm -rf ./webapps/*
ARG JAR_FILE=*.war
COPY ${JAR_FILE} ./webapps/edu-msa-board-1.0.0.war

EXPOSE 28082
```

- ✓ **FROM**
컨테이너 기반이 되는 베이스 이미지
- ✓ **WORKDIR**
명령을 실행할 작업 디렉토리 설정
- ✓ **COPY**
컨테이너 빌드 시 컨테이너로 파일 복사

» Dockerfile 작성

도커 파일 예시 (2/3)

```
FROM tomcat:9-jre8-alpine
WORKDIR /usr/local/tomcat
COPY server.xml ./conf
RUN rm -rf ./webapps/*
ARG JAR_FILE=*.war
COPY ${JAR_FILE} ./webapps/edu-msa-board-1.0.0.war

EXPOSE 28082
```

- ✓ RUN
베이스 이미지에서 실행할 명령
- ✓ ARG
파일에 사용할 변수 값 설정
- ✓ EXPOSE
외부에 노출시킬 포트 번호

» Dockerfile 작성

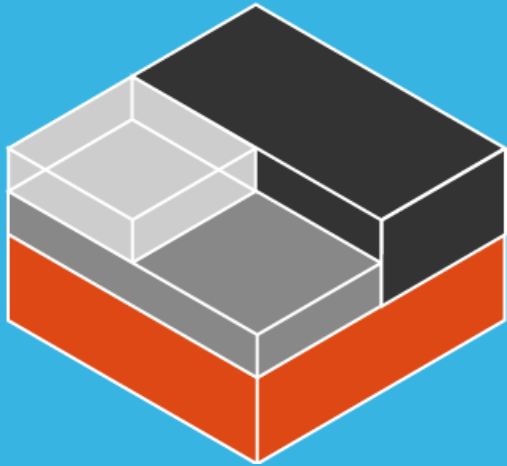
도커 파일 예시 (3/3)

```
FROM openjdk:8-jdk-alpine
ARG JAR_FILE=edu-msa-zuul-1.0.0.war
COPY ${JAR_FILE} edu-msa-zuul-1.0.0.war
COPY application.yml /application.yml
ENTRYPOINT ["java","-jar","-Dspring.config.location=application.yml","/edu-msa-zuul-1.0.0.war"]
```

✓ ENTRYPOINT

컨테이너 실행 시 수행되는 명령어를 지정(프로파일 값 변경)

» LXC VS Docker



LXC

- ✓ 컨테이너가 여러 프로세스 실행

VS



Docker

- ✓ 컨테이너가 단일 프로세스 실행

» LXC와 Docker 비교

LXC

- ✓ 컨테이너 하나가 여러 프로세스 실행
- ✓ 하나의 컨테이너에 다양한 애플리케이션을 실행
- ✓ 다중 프로세스를 관리하는 init 프로세스를 추가하여 여러 프로세스를 실행 할 수 있음

도커

- ✓ 컨테이너 하나가 단일 프로세스 실행
- ✓ 하나의 컨테이너에 하나의 애플리케이션만 실행
- ✓ 하나의 애플리케이션만 실행시키기 때문에 가볍고 실행 속도가 빠름
- ✓ 시스템에 대한 부담이 적음
- ✓ 컨테이너의 이식성이 뛰어남

» Docker CLI 명령어 사용법

기본 형태

- ✓ Docker CLI: <https://docs.docker.com/engine/reference/commandline/cli/>

기본 Syntax

```
$ docker [COMMAND] [OPTIONS] [ARG...]
```

설명

- ✓ [COMMAND]
수행하려는 동작 (search, pull, images, run, ps, stop, start, exec, rm, rmi, inspect 등)
- ✓ [OPTIONS]
명령의 세부 기능 (-d, -p, -v, -f, -name, --limit 등)
- ✓ [ARG...]
명령으로 전달 되는 값

» Docker CLI 명령어 사용법

이미지 검색

기본 Syntax

`$ docker [COMMAND] [IMAGE] [OPTIONS]`

설명

도커 허브에서 이미지 검색 (옵션 `--limit 6`는 6개 까지만 검색)

사용 예시

`$ docker search nginx --limit 6`

```

l:~docker>docker search nginx --limit 6
NAME                DESCRIPTION                STARS     OFFICIAL   AUTOMATED
nginx               Official build of Nginx.    15719     [OK]
jwilder/nginx-proxy Automated Nginx reverse proxy for docker con... 2088      [OK]
jlesage/nginx-proxy-manager Docker container for Nginx Proxy Manager      142      [OK]
nginxdemos/hello    NGINX webserver that serves a simple page co... 75        [OK]
nginx/nginx-ingress NGINX and NGINX Plus Ingress Controllers fo... 55
nginxinc/nginx-unprivileged Unprivileged NGINX Dockerfiles                54

```

02 Docker CLI 명령어 이해

» Docker CLI 명령어 사용법

이미지 다운로드

기본 Syntax

\$ docker [COMMAND] [IMAGE]:[TAG]

설명

도커 허브에서 이미지 다운로드 (버전을 변경 시 [TAG] 부분 변경)

사용 예시

\$ docker pull nginx:latest

```
l:\w\docker>docker pull nginx:latest
latest: Pulling from library/nginx
b380bbd43752: Pull complete
fca7e12d1754: Pull complete
745ab57616cb: Pull complete
a4723e260b6f: Pull complete
1c84ebdff681: Pull complete
858292fd2e56: Pull complete
Digest: sha256:644a70516a26004c97d0d85c7fe1d0c3a67ea8ab7ddf4aff193d9f301670cf36
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

02 Docker CLI 명령어 이해

» Docker CLI 명령어 사용법

이미지 조회

기본 Syntax	\$ docker [COMMAND]
-----------	---------------------

설명	도커 허브에서 다운 받은 이미지를 로컬에서 조회
----	----------------------------

사용 예시	\$ docker images
-------	------------------

```
l:~$ docker> docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    87a94228f133   2 weeks ago   133MB
```

» Docker CLI 명령어 사용법

이미지 상세 조회

기본 Syntax

`$ docker [COMMAND] [IMAGE]`

설명

도커 허브에서 다운 받은 이미지를 로컬에서 상세 조회

사용 예시

`$ docker inspect nginx`

```
I:\#docker>docker inspect nginx
[
  {
    "Id": "sha256:87a94228f133e2da99cb16d653cd1373c5b4e8689956386c1c12b60a20421a02",
    "RepoTags": [
      "nginx:latest"
    ],
    "RepoDigests": [
      "nginx@sha256:644a70516a26004c97d0d85c7fe1d0c3a67ea8ab7ddf4aff193d9f301670cf36"
    ],
    "Parent": "",
    "Comment": "",
    "Created": "2021-10-12T02:03:40.360294686Z",
    "Container": "21fd1c6cb532225ca7e04c77f6592e220574b919aec07021663576ef438e0fee",
```


» Docker CLI 명령어 사용법

컨테이너 실행

기본 Syntax

`$ docker [COMMAND] [OPTIONS] [IMAGE]`

설명

도커 이미지를 컨테이너로 생성하고 실행
(옵션 `-p`는 호스트와 컨테이너의 포트를 연결)

사용 예시

`$ docker run -p 80:80 nginx`

```
l:~$ docker>docker run -p 80:80 nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2021/10/29 01:24:35 [notice] 1#1: using the "epoll" event method
2021/10/29 01:24:35 [notice] 1#1: nginx/1.21.3
2021/10/29 01:24:35 [notice] 1#1: built by gcc 8.3.0 (Debian 8.3.0-6)
2021/10/29 01:24:35 [notice] 1#1: OS: Linux 5.10.16.3-microsoft-standard-WSL2
2021/10/29 01:24:35 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2021/10/29 01:24:35 [notice] 1#1: start worker processes
```

» Docker CLI 명령어 사용법

컨테이너 조회

기본 Syntax

\$ docker [COMMAND]

설명

실행 중인 도커 컨테이너 확인

사용 예시

\$ docker ps

```

l:~#docker>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                               NAMES
f21617291c19   nginx    "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   0.0.0.0:80->80/tcp, :::80->80/tcp   exciting_feynman

```

» Docker CLI 명령어 사용법

컨테이너 종료

기본 Syntax

```
$ docker [COMMAND] [CONTAINER_NAME]
```

설명

실행 중인 도커 컨테이너 종료

사용 예시

```
$ docker stop exciting_feynman
```

```
l:~$ docker> docker stop exciting_feynman
exciting_feynman
```

```
l:~$ docker> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED   STATUS    PORTS   NAMES
```

» Docker CLI 명령어 사용법

컨테이너 실행

기본 Syntax

```
$ docker [COMMAND] [CONTAINER_NAME]
```

설명

중단되어 있는 도커 컨테이너 실행

사용 예시

```
$ docker start exciting_feynman
```

```
l:\#docker>docker start exciting_feynman
exciting_feynman
```

```
l:\#docker>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
f21617291c19   nginx    "/docker-entrypoint.…"   10 minutes ago Up 2 seconds  0.0.0.0:80->80/tcp, :::80->80/tcp  exciting_feynman
```

» Docker CLI 명령어 사용법

컨테이너 삭제

기본 Syntax

`docker [COMMAND] [OPTIONS] [CONTAINER_NAME]`

설명

실행 중인 도커 컨테이너 삭제

사용 예시

`docker rm -f exciting_feynman`

```
I:\#docker>docker rm -f exciting_feynman
exciting_feynman
```

```
I:\#docker>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
```

» Docker CLI 명령어 사용법

이미지 삭제

기본 Syntax

```
$ docker [COMMAND] [OPTIONS] [IMAGE_ID]
```

설명

이미지 삭제 (옵션 -f는 강제로 이미지 삭제)

사용 예시

```
$ docker rmi -f 87a94228f133
```

```
l:~#docker>docker rmi -f 87a94228f133
Untagged: nginx:latest
Untagged: nginx@sha256:644a70516a26004c97d0d85c7fe1d0c3a67ea8ab7ddf4aff193d9f301670cf36
Deleted: sha256:87a94228f133e2da99cb16d653cd1373c5b4e8689956386c1c12b60a20421a02

l:~#docker>docker images
REPOSITORY    TAG        IMAGE ID    CREATED    SIZE
```

» Docker CLI 명령어 사용법

로그인

기본 Syntax

```
$ docker [COMMAND] [OPTIONS] [DOCKER_HUB_ID]
```

설명

도커 로그인

사용 예시

```
$ docker login -u paastakr
```

```
l:~$ docker> docker login -u paastakr
Password:
Login Succeeded
```

» Docker CLI 명령어 사용법

이미지 빌드

기본 Syntax

`$ docker [COMMAND] [OPTIONS] [IMAGE_NAME]:[TAG] [POSITION]`

설명

이미지 빌드 (옵션 `--tag`는 이미지 이름과 태그를 설정)

사용 예시

`$ docker build --tag edu-msa-board:latest .`

```

[!:\docker\board>docker build --tag edu-msa-board:latest .
[+] Building 3.2s (11/11) FINISHED
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 221B                                              0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [internal] load metadata for docker.io/library/tomcat:9-jre8-alpine          2.6s
=> [auth] library/tomcat:pull token for registry-1.docker.io                   0.0s
=> [internal] load build context                                                0.2s
=> => transferring context: 22.59MB                                             0.2s
=> [1/5] FROM docker.io/library/tomcat:9-jre8-alpine@sha256:17accf0afeeccc0310d363490cd60a788aa4630ab9c9c802231 0.0s
=> CACHED [2/5] WORKDIR /usr/local/tomcat                                     0.0s
=> CACHED [3/5] COPY server.xml ./conf                                         0.0s
=> CACHED [4/5] RUN rm -rf ./webapps/*                                          0.0s

```


02 Docker CLI 명령어 이해

» Docker CLI 명령어 사용법

이미지 저장

기본 Syntax

```
$ docker [COMMAND] [OPTIONS] [FILE_NAME]  
[IMAGE_NAME]:[TAG]
```

설명

빌드한 이미지 저장 (옵션 -o는 빌드한 이미지를 tar 파일로 로컬에 저장)

사용 예시

```
$ docker save -o edu-msa-board.tar edu-msa-board:latest
```

```
l:~#docker#board>docker save -o edu-msa-board.tar edu-msa-board:latest
```

» Docker CLI 명령어 사용법

이미지 로드

기본 Syntax

```
$ docker [COMMAND] [OPTIONS] [FILE_NAME]
```

설명

로컬 이미지를 로드 (옵션 -i 는 압축 파일에서 이미지를 읽음)

사용 예시

```
$ docker load -i edu-msa-board.tar
```

```
l:~#docker#edu-msa-board>docker load -i edu-msa-board.tar
Loaded image: edu-msa-board:latest
```

» Docker CLI 명령어 사용법

이미지 태그

기본 Syntax

```
$ docker [COMMAND] [BEFORE_IMAGE_NAME]  
[DOCKER_HUB_ID]/[IMAGE_NAME]
```

설명

기존에 빌드한 이미지의 이름을 이미지 저장소에 업로드할 이미지 이름으로 변경

사용 예시

```
$ docker tag edu-msa-board paastakr/edu-msa-board
```

```
l:~#docker#board>docker tag edu-msa-board paastakr/edu-msa-board
```

02 Docker CLI 명령어 이해

» Docker CLI 명령어 사용법

이미지 업로드

기본 Syntax

\$ docker [COMMAND] [DOCKER_HUB_ID]/[IMAGE_NAME]

설명

이미지 업로드

사용 예시

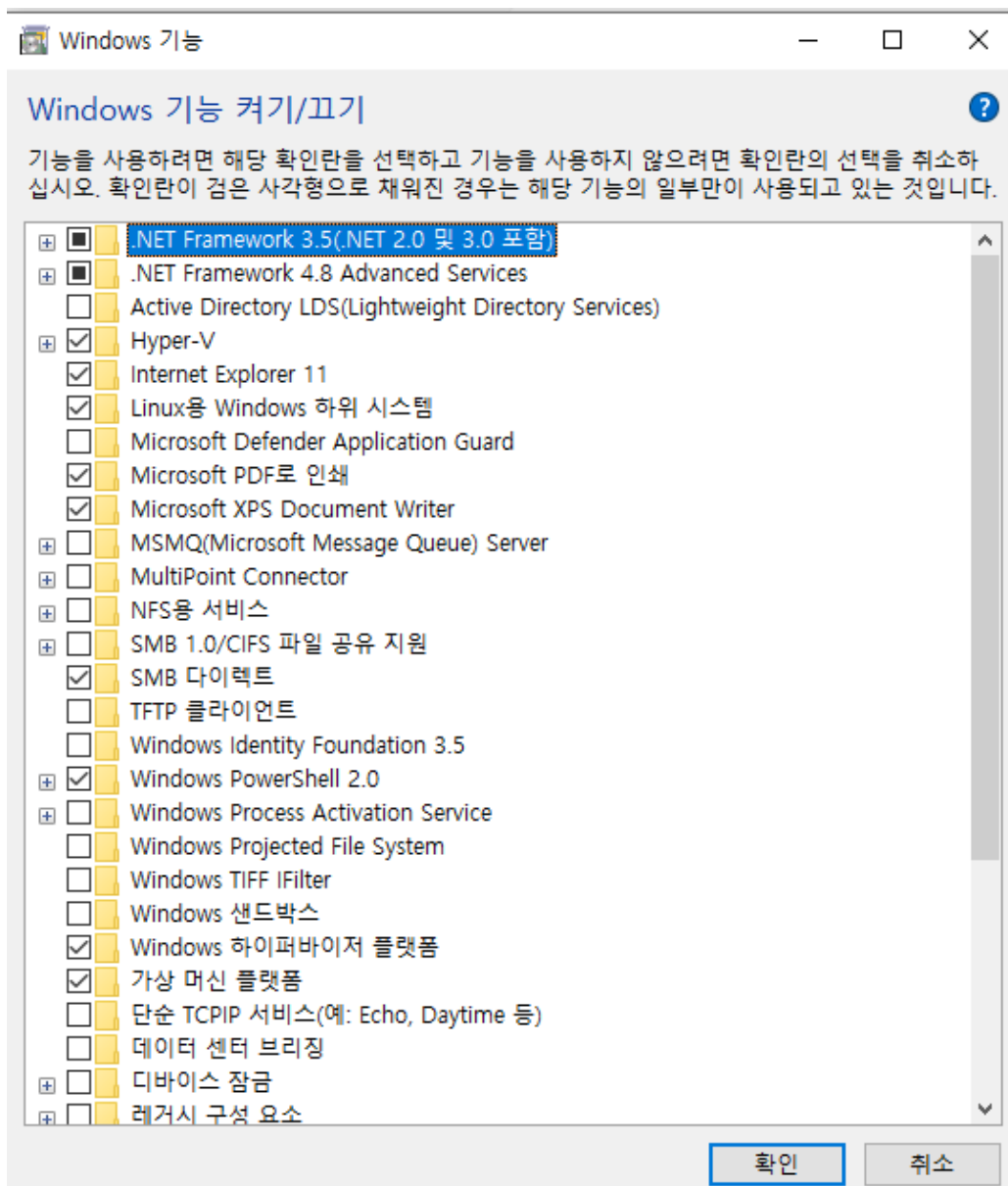
\$ docker push paastakr/edu-msa-board

```
l:\w\docker\wboard>docker push paastakr/edu-msa-board
Using default tag: latest
The push refers to repository [docker.io/paastakr/edu-msa-board]
c6ce6e8a6d88: Pushed
f5c0d88f8c41: Pushed
2a315d780e2a: Pushed
5f70bf18a086: Pushed
b7d850202de0: Pushed
d1d0b1719b96: Pushed
48988bb7b861: Pushed
edd61588d126: Pushed
9b9b7f3d56a0: Pushed
```

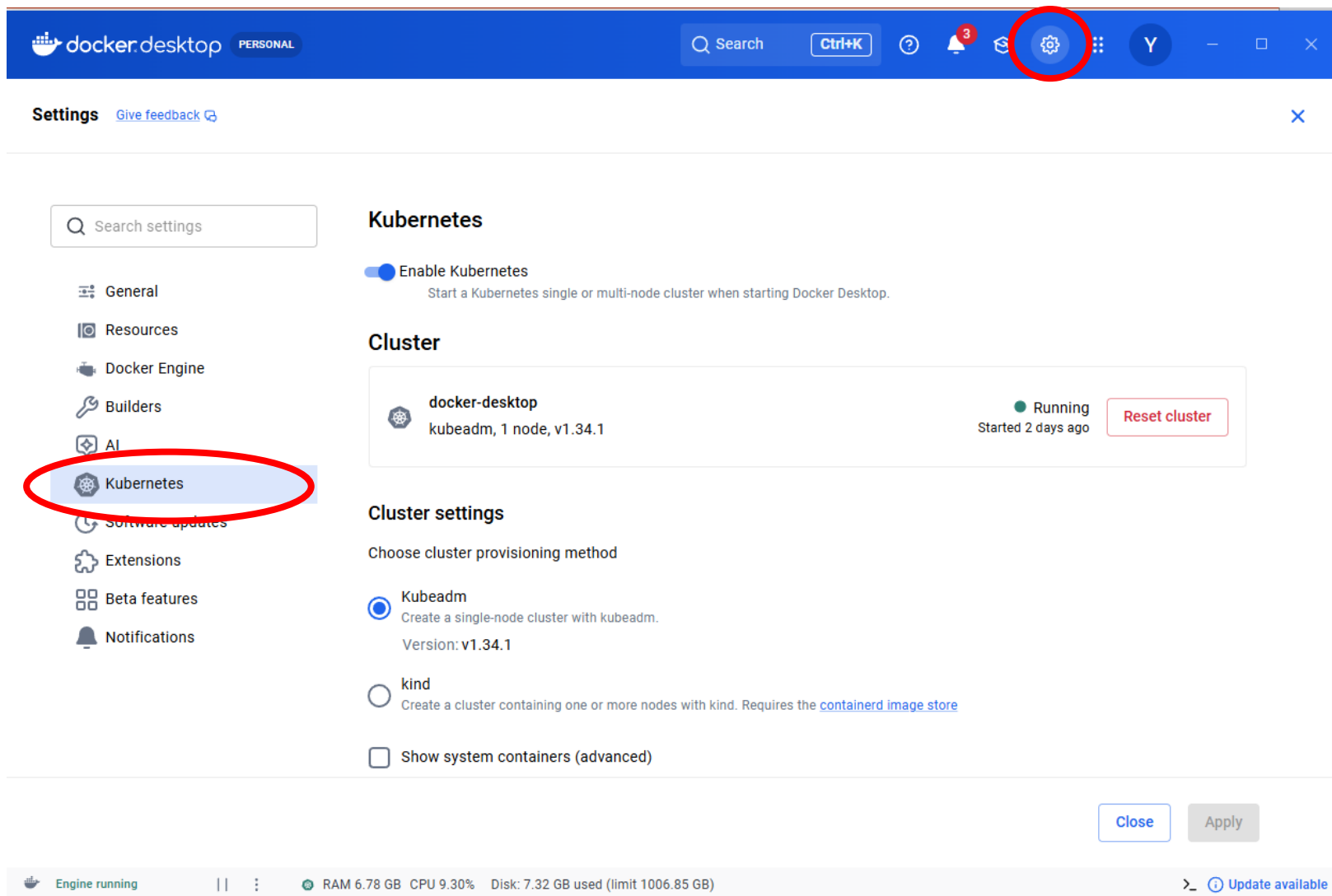
» Docker 데스크탑 설치 조건

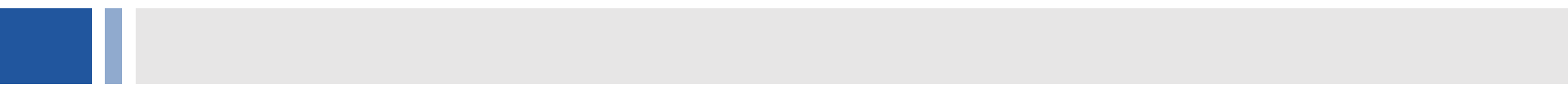
1. 운영체제(OS) 확인

- Docker Desktop은 특정 Windows 버전 이상에서만 정상 동작
- 필수 조건
 - Windows 10 64bit Pro / Enterprise / Education (21H2 이상) 또는 Windows 11
 - 가상화 기능이 체크되어 있어야 함
 - 가상화 기능위에서 WSL2를 활용해 도커 데스크탑이 기동하는 구조임



» Docker 데스크탑 쿠버네티스 활성화





감사합니다

THANK YOU